Arrowhead core systems in Rust

Native implementation and interface catalogue

Abstract

Five active core modules in the local Java 5.2.2 reference are implemented as independently running native Rust systems. This catalogue records the delivered design, services, trust boundaries, interfaces and remaining work. It does not certify full Java interchangeability or a modified cloud deployment.

> **Release boundary.** Do not replace a Java deployment based only on route coverage. Review the compatibility and verification records, close the documented feature gaps, and qualify real-cloud interoperability, state migration and rollback.

Contents

1 ServiceRegistry **3**

2 Authentication **5**

3 ConsumerAuthorization **7**

4 DynamicServiceOrchestration **9**

5 SimpleStoreServiceOrchestration **11**

6 Verification and release gates **13**

1 ServiceRegistry

1.1 System overview

Native Rust implementation of the local Java module `serviceregistry/`. Each named executable checks its configured role and owns an independent identity, listen port and state file. Shared Rust modules implement the runtime; no Java, Python or Go process implements the service.

1.2 Produced and consumed services

Produced HTTPS definitions: `deviceDiscovery`, `echo`, `generalManagement`, `serviceDiscovery`, `serviceRegistryManagement`, `systemDiscovery`. Service versions are 1.0.0; the target system version is 5.2.2. Authorities consumed as applicable are Authentication, ServiceRegistry and ConsumerAuthorization. Explicit pinned HTTPS origins and valid cloud identities must be provisioned.

1.3 Architecture and persistent state

The listener uses Tokio, Hyper/Axum and rustls with mandatory client certificates. Verified peer identity is supplied through a private request extension, never a declared-identity header. Business modules share locked JSON state with clone/validate/fsync/rename transactions. Credentials and keys are separate owner-only files. Cross-system operations and callbacks are not distributed transactions.

1.4 Security

Local cloud CN/profile, certificate chain and validity are verified. Identity tokens are checked against Authentication and bound to the client certificate. Management requires a verified sysop or explicit broad whitelist. Outbound peer chain/SAN and expected system CN/profile are checked before credentials are sent. Optional PEM CRLs are enforced at startup-configured trust boundaries and require process restart to reload. No plaintext HTTP listener, TLS bypass or declared SYSTEM identity mode is provided.

1.5 Implementation status and delimitations

This is a development profile, not a certified interchangeable Java replacement. HTTP route presence is not semantic equivalence. Java database migration, MQTT application APIs and production qualification remain outstanding. Registry records, interface templates, expiry, cascades, authorized discovery and management are implemented. Java SQL import, the full property-validator catalogue, normalization, advanced version/-time/address matching and all discovery-policy options need additional work. Unsupported custom validators fail explicitly; they are not silently accepted.

1.6 HTTPS interface operations

HTTPS uses JSON unless an operation specifies text. Request bodies are limited to 1 MiB; upstream responses to 8 MiB. Identity-token format: `Bearer IDENTITY-TOKEN//token`. Certificate-only identity operations still require verified mutual TLS and apply credential/certificate binding.

Method	Path	Access
GET	/serviceregistry/general/mgmt/get-config	management
POST	/serviceregistry/general/mgmt/logs	management
GET	/serviceregistry/monitor/echo	certificate
POST	/serviceregistry/device-discovery/register	identity
POST	/serviceregistry/device-discovery/lookup	identity
DELETE	/serviceregistry/device-discovery/revoke/{name}	identity
POST	/serviceregistry/system-discovery/register	identity
POST	/serviceregistry/system-discovery/lookup	identity
DELETE	/serviceregistry/system-discovery/revoke	identity
POST	/serviceregistry/service-discovery/register	identity
POST	/serviceregistry/service-discovery/lookup	identity
DELETE	/serviceregistry/service-discovery/revoke/{instanceId}	identity
POST	/serviceregistry/mgmt/devices/query	management
POST	/serviceregistry/mgmt/devices	management
DELETE	/serviceregistry/mgmt/devices	management
PUT	/serviceregistry/mgmt/devices	management
POST	/serviceregistry/mgmt/systems/query	management
POST	/serviceregistry/mgmt/systems	management
DELETE	/serviceregistry/mgmt/systems	management
PUT	/serviceregistry/mgmt/systems	management
POST	/serviceregistry/mgmt/service-definitions/query	management
POST	/serviceregistry/mgmt/service-definitions	management
DELETE	/serviceregistry/mgmt/service-definitions	management
POST	/serviceregistry/mgmt/service-instances/query	management
POST	/serviceregistry/mgmt/service-instances	management
DELETE	/serviceregistry/mgmt/service-instances	management
PUT	/serviceregistry/mgmt/service-instances	management
POST	/serviceregistry/mgmt/interface-templates/query	management
POST	/serviceregistry/mgmt/interface-templates	management
DELETE	/serviceregistry/mgmt/interface-templates	management

Errors contain errorMessage, errorCode, exceptionType, origin and timestamp. Exact validation/error parity and all Java security-policy settings remain release gates. Unknown routes return 404; unavailable authorities fail closed.

2 Authentication

2.1 System overview

Native Rust implementation of the local Java module `authentication/`. Each named executable checks its configured role and owns an independent identity, listen port and state file. Shared Rust modules implement the runtime; no Java, Python or Go process implements the service.

2.2 Produced and consumed services

Produced HTTPS definitions: `echo`, `generalManagement`, `identity`, `identityManagement`. Service versions are 1.0.0; the target system version is 5.2.2. Authorities consumed as applicable are Authentication, ServiceRegistry and ConsumerAuthorization. Explicit pinned HTTPS origins and valid cloud identities must be provisioned.

2.3 Architecture and persistent state

The listener uses Tokio, Hyper/Axum and rustls with mandatory client certificates. Verified peer identity is supplied through a private request extension, never a declared-identity header. Business modules share locked JSON state with clone/validate/fsync/rename transactions. Credentials and keys are separate owner-only files. Cross-system operations and callbacks are not distributed transactions.

2.4 Security

Local cloud CN/profile, certificate chain and validity are verified. Identity tokens are checked against Authentication and bound to the client certificate. Management requires a verified sysop or explicit broad whitelist. Outbound peer chain/SAN and expected system CN/profile are checked before credentials are sent. Optional PEM CRLs are enforced at startup-configured trust boundaries and require process restart to reload. No plaintext HTTP listener, TLS bypass or declared SYSTEM identity mode is provided.

2.5 Implementation status and delimitations

This is a development profile, not a certified interchangeable Java replacement. HTTP route presence is not semantic equivalence. Java database migration, MQTT application APIs and production qualification remain outstanding. PASSWORD identities use bcrypt and single active opaque sessions. Java SQL/session import, complete query/time normalization and alternative framework transports require further work. No stored plaintext password or raw identity token is used.

2.6 HTTPS interface operations

HTTPS uses JSON unless an operation specifies text. Request bodies are limited to 1 MiB; upstream responses to 8 MiB. Identity-token format: `Bearer IDENTITY-TOKEN//token`. Certificate-only identity operations still require verified mutual TLS and apply credential/certificate binding.

Method	Path	Access
GET	/authentication/general/mgmt/get-config	management
POST	/authentication/general/mgmt/logs	management
GET	/authentication/monitor/echo	certificate
POST	/authentication/identity/login	certificate
POST	/authentication/identity/logout	certificate
POST	/authentication/identity/change	certificate
GET	/authentication/identity/verify/{token}	identity
POST	/authentication/mgmt/identities	management
PUT	/authentication/mgmt/identities	management
DELETE	/authentication/mgmt/identities	management
POST	/authentication/mgmt/identities/query	management
DELETE	/authentication/mgmt/sessions	management
POST	/authentication/mgmt/sessions	management

Errors contain errorMessage, errorCode, exceptionType, origin and timestamp. Exact validation/error parity and all Java security-policy settings remain release gates. Unknown routes return 404; unavailable authorities fail closed.

3 ConsumerAuthorization

3.1 System overview

Native Rust implementation of the local Java module `consumer-authorization/`. Each named executable checks its configured role and owns an independent identity, listen port and state file. Shared Rust modules implement the runtime; no Java, Python or Go process implements the service.

3.2 Produced and consumed services

Produced HTTPS definitions: `authorization`, `authorizationManagement`, `authorizationToken`, `authorizationTokenManagement`, `echo`, `generalManagement`. Service versions are 1.0.0; the target system version is 5.2.2. Authorities consumed as applicable are Authentication, ServiceRegistry and ConsumerAuthorization. Explicit pinned HTTPS origins and valid cloud identities must be provisioned.

3.3 Architecture and persistent state

The listener uses Tokio, Hyper/Axum and rustls with mandatory client certificates. Verified peer identity is supplied through a private request extension, never a declared-identity header. Business modules share locked JSON state with clone/validate/fsync/rename transactions. Credentials and keys are separate owner-only files. Cross-system operations and callbacks are not distributed transactions.

3.4 Security

Local cloud CN/profile, certificate chain and validity are verified. Identity tokens are checked against Authentication and bound to the client certificate. Management requires a verified sysop or explicit broad whitelist. Outbound peer chain/SAN and expected system CN/profile are checked before credentials are sent. Optional PEM CRLs are enforced at startup-configured trust boundaries and require process restart to reload. No plaintext HTTP listener, TLS bypass or declared SYSTEM identity mode is provided.

3.5 Implementation status and delimitations

This is a development profile, not a certified interchangeable Java replacement. HTTP route presence is not semantic equivalence. Java database migration, MQTT application APIs and production qualification remain outstanding. Management-over-provider policy precedence, scoped policies, metadata checks, usage/time-limited tokens, Base64 and RSA JWT tokens, AES wire encryption and authenticated key wrapping are implemented. Foreign-consumer/federation token generation, Java database import, complete DTO/error parity and broad cryptographic interoperability fixtures remain incomplete. AES ECB/CBC are legacy Java wire formats, not recommendations for new protocols.

3.6 HTTPS interface operations

HTTPS uses JSON unless an operation specifies text. Request bodies are limited to 1 MiB; upstream responses to 8 MiB. Identity-token format: `Bearer IDENTITY-TOKEN//token`. Certificate-only identity operations still require verified mutual TLS and apply credential/certificate binding.

Method	Path	Access
GET	/consumerauthorization/general/mgmt/get-config	management
POST	/consumerauthorization/general/mgmt/logs	management
GET	/consumerauthorization/monitor/echo	certificate
POST	/consumerauthorization/authorization/grant	identity
DELETE	/consumerauthorization/authorization/revoke/{instanceId}	identity
POST	/consumerauthorization/authorization/lookup	identity
POST	/consumerauthorization/authorization/verify	identity
POST	/consumerauthorization/authorization/mgmt/grant	management
DELETE	/consumerauthorization/authorization/mgmt/revoke	management
POST	/consumerauthorization/authorization/mgmt/query	management
POST	/consumerauthorization/authorization/mgmt/check	management
POST	/consumerauthorization/authorization-token/generate	identity
GET	/consumerauthorization/authorization-token/token/verify/{token}	identity
GET	/consumerauthorization/authorization-token/public-key	identity
POST	/consumerauthorization/authorization-token/encryption-key	identity
DELETE	/consumerauthorization/authorization-token/encryption-key	identity
POST	/consumerauthorization/authorization/mgmt/token/generate	management
POST	/consumerauthorization/authorization/mgmt/token/query	management
DELETE	/consumerauthorization/authorization/mgmt/token/revoke	management
POST	/consumerauthorization/authorization/mgmt/token/encryption-key	management
DELETE	/consumerauthorization/authorization/mgmt/token/encryption-key	management

Errors contain errorMessage, errorCode, exceptionType, origin and timestamp. Exact validation/error parity and all Java security-policy settings remain release gates. Unknown routes return 404; unavailable authorities fail closed.

4 DynamicServiceOrchestration

4.1 System overview

Native Rust implementation of the local Java module `serviceorchestration-dynamic/`. Each named executable checks its configured role and owns an independent identity, listen port and state file. Shared Rust modules implement the runtime; no Java, Python or Go process implements the service.

4.2 Produced and consumed services

Produced HTTPS definitions: `generalManagement`, `serviceOrchestration`, `serviceOrchestrationHistoryManagement`, `serviceOrchestrationLockManagement`, `serviceOrchestrationPushManagement`. Service versions are 1.0.0; the target system version is 5.2.2. Authorities consumed as applicable are Authentication, ServiceRegistry and ConsumerAuthorization. Explicit pinned HTTPS origins and valid cloud identities must be provisioned.

4.3 Architecture and persistent state

The listener uses Tokio, Hyper/Axum and rustls with mandatory client certificates. Verified peer identity is supplied through a private request extension, never a declared-identity header. Business modules share locked JSON state with clone/validate/fsync/rename transactions. Credentials and keys are separate owner-only files. Cross-system operations and callbacks are not distributed transactions.

4.4 Security

Local cloud CN/profile, certificate chain and validity are verified. Identity tokens are checked against Authentication and bound to the client certificate. Management requires a verified sysop or explicit broad whitelist. Outbound peer chain/SAN and expected system CN/profile are checked before credentials are sent. Optional PEM CRLs are enforced at startup-configured trust boundaries and require process restart to reload. No plaintext HTTP listener, TLS bypass or declared SYSTEM identity mode is provided.

4.5 Implementation status and delimitations

This is a development profile, not a certified interchangeable Java replacement. HTTP route presence is not semantic equivalence. Java database migration, MQTT application APIs and production qualification remain outstanding. Local discovery, authorization, operation filtering, token assignment, preferred providers, matchmaking, exclusive locks, history and HTTPS push are implemented. Inter-cloud, translation, QoS, complete blacklist integrations and automatic registry-event-driven reevaluation are not ported; requested unsupported integrations fail with 501. Do not replace working federation with this profile.

4.6 HTTPS interface operations

HTTPS uses JSON unless an operation specifies text. Request bodies are limited to 1 MiB; upstream responses to 8 MiB. Identity-token format: `Bearer IDENTITY-TOKEN//token`. Certificate-only identity operations still require verified mutual TLS and apply credential/certificate binding.

Method	Path	Access
GET	/serviceorchestration/general/mgmt/get-config	management
POST	/serviceorchestration/general/mgmt/logs	management
POST	/serviceorchestration/orchestration/pull	identity
POST	/serviceorchestration/orchestration/subscribe	identity
DELETE	/serviceorchestration/orchestration/unsubscribe/{id}	identity
POST	/serviceorchestration/orchestration/mgmt/push/subscribe	management
POST	/serviceorchestration/orchestration/mgmt/push/trigger	management
DELETE	/serviceorchestration/orchestration/mgmt/push/unsubscribe	management
POST	/serviceorchestration/orchestration/mgmt/push/query	management
POST	/serviceorchestration/orchestration/mgmt/history/query	management
POST	/serviceorchestration/orchestration/mgmt/lock/create	management
POST	/serviceorchestration/orchestration/mgmt/lock/query	management
DELETE	/serviceorchestration/orchestration/mgmt/lock/remove/{owner}	management

Errors contain errorMessage, errorCode, exceptionType, origin and timestamp. Exact validation/error parity and all Java security-policy settings remain release gates. Unknown routes return 404; unavailable authorities fail closed.

5 SimpleStoreServiceOrchestration

5.1 System overview

Native Rust implementation of the local Java module `serviceorchestration-simple/`. Each named executable checks its configured role and owns an independent identity, listen port and state file. Shared Rust modules implement the runtime; no Java, Python or Go process implements the service.

5.2 Produced and consumed services

Produced HTTPS definitions: `generalManagement`, `serviceOrchestration`, `serviceOrchestrationHistoryManagement`, `serviceOrchestrationPushManagement`, `serviceOrchestrationSimpleStoreManagement`. Service versions are 1.0.0; the target system version is 5.2.2. Authorities consumed as applicable are Authentication, ServiceRegistry and ConsumerAuthorization. Explicit pinned HTTPS origins and valid cloud identities must be provisioned.

5.3 Architecture and persistent state

The listener uses Tokio, Hyper/Axum and rustls with mandatory client certificates. Verified peer identity is supplied through a private request extension, never a declared-identity header. Business modules share locked JSON state with clone/validate/fsync/rename transactions. Credentials and keys are separate owner-only files. Cross-system operations and callbacks are not distributed transactions.

5.4 Security

Local cloud CN/profile, certificate chain and validity are verified. Identity tokens are checked against Authentication and bound to the client certificate. Management requires a verified sysop or explicit broad whitelist. Outbound peer chain/SAN and expected system CN/profile are checked before credentials are sent. Optional PEM CRLs are enforced at startup-configured trust boundaries and require process restart to reload. No plaintext HTTP listener, TLS bypass or declared SYSTEM identity mode is provided.

5.5 Implementation status and delimitations

This is a development profile, not a certified interchangeable Java replacement. HTTP route presence is not semantic equivalence. Java database migration, MQTT application APIs and production qualification remain outstanding. Stored candidate priority selection, preferred providers, matchmaking, history, subscriptions and explicit HTTPS push jobs are implemented. Store results intentionally retain Java's sparse result shape. Full Java query/priority-normalization semantics, event-driven reevaluation, MQTT callbacks and Java state import are not ported. Unsupported simple-store criteria are reported as ignored warnings.

5.6 HTTPS interface operations

HTTPS uses JSON unless an operation specifies text. Request bodies are limited to 1 MiB; upstream responses to 8 MiB. Identity-token format: `Bearer IDENTITY-TOKEN//token`. Certificate-only identity operations still require verified mutual TLS and apply credential/certificate binding.

Method	Path	Access
GET	/serviceorchestration/general/mgmt/get-config	management
POST	/serviceorchestration/general/mgmt/logs	management
POST	/serviceorchestration/orchestration/pull	identity
POST	/serviceorchestration/orchestration/subscribe	identity
DELETE	/serviceorchestration/orchestration/unsubscribe/{id}	identity
POST	/serviceorchestration/orchestration/mgmt/push/subscribe	management
POST	/serviceorchestration/orchestration/mgmt/push/trigger	management
DELETE	/serviceorchestration/orchestration/mgmt/push/unsubscribe	management
POST	/serviceorchestration/orchestration/mgmt/push/query	management
POST	/serviceorchestration/orchestration/mgmt/history/query	management
POST	/serviceorchestration/orchestration/mgmt/simple-store/create	management
POST	/serviceorchestration/orchestration/mgmt/simple-store/query	management
PUT	/serviceorchestration/orchestration/mgmt/simple-store/modify-priorities	management
DELETE	/serviceorchestration/orchestration/mgmt/simple-store/remove	management

Errors contain errorMessage, errorCode, exceptionType, origin and timestamp. Exact validation/error parity and all Java security-policy settings remain release gates. Unknown routes return 404; unavailable authorities fail closed.

6 Verification and release gates

The verification record distinguishes builds, unit and isolated HTTPS tests, source-route coverage and real deployment qualification. Required outstanding work includes Java-versus-Rust golden DTO/error/security fixtures, completion of unsupported integrations, real brokers and federation peers, imported database recovery, backup/restore and rollback. No document declares a production-ready replacement.